

EE 446 HW 1

Aarav Wadhwani | aaravw28@uw.edu | 2363021

Problem 1 - Part (e)

1. **Analyze and compare** the results from previous parts: Which model had the smallest size? Which performed best?

Among the single-technique compressions tested in parts (b), (c), and (d), Full Integer (INT8) Quantization yielded the smallest model at 5.74 KB, while perfectly maintaining the baseline accuracy of 1.00. Knowledge Distillation also significantly reduced the size to 6.10 KB but resulted in a slight accuracy drop to 0.89. Pruning alone did not reduce the flat TFLite file size (14.14 KB) without secondary compression.

2. **Propose a strategy** that combines or enhances techniques learned so far.

To achieve further size reduction, the best approach is to layer orthogonal compression techniques. Specifically, applying Full Integer (INT8) Quantization to the Knowledge Distilled Student Model. Distillation provides an architectural reduction (fewer total parameters), while quantization provides a precision reduction (reducing the remaining parameters from 32-bit floats to 8-bit integers).

3. **Implement** your proposed solution.

```
# <-- (if needed) Enter your code here <--#
quantize_and_evaluate(student_model, X_test_scaled, y_test_cat, "int8", "model_kd_int8.tflite")
```

4. **Evaluate** the resulting model using both:

```
INT8 TFLite model size: 3.62 KB
      precision    recall  f1-score   support

      0       0.89       0.94       0.92         18
      1       0.86       0.86       0.86         21
      2       0.93       0.87       0.90         15

   accuracy                   0.89         54
  macro avg       0.89       0.89       0.89         54
 weighted avg       0.89       0.89       0.89         54

[[17  1  0]
 [ 2 18  1]
 [ 0  2 13]]
```

5. Justify your results:

Further size reduction was successfully achieved by combining techniques. By quantizing the distilled student model, the final memory footprint dropped to 3.62 KB, making it roughly 74% smaller than the baseline 14.07 KB model. The change that made the biggest difference was moving from 32-bit floating-point weights to 8-bit integer weights on an already condensed architecture. Because the student model's parameters were highly optimized to mimic the teacher's soft labels, the model was robust enough to handle the precision loss of INT8 quantization without sacrificing any additional classification performance beyond the initial distillation drop.

Problem 2:

1. Learning Blocks in the Free Tier

- Classification (Supervised Learning): Learns to categorize input data into distinct classes.

Example Model: An audio classifier that identifies the sounds of different environmental noises (e.g., glass breaking, alarms, background noise).

- Regression (Supervised Learning): Predicts continuous numerical values rather than discrete classes.

Example Model: A model that predicts the exact RPM of a motor based on high-frequency vibration sensor data.

- Anomaly Detection (Unsupervised Learning / Clustering): Identifies data points that fall outside the normal, expected distribution of training data.

Example Model: A K-Means clustering model that flags unusual machine vibrations as "anomalies" to predict predictive maintenance needs.

- Object Detection (Supervised Learning): Identifies and locates multiple objects within a single image.

Example Model: A FOMO (Faster Objects, More Objects) model designed to count the number of defective items moving down a conveyor belt.

- Keyword Spotting (Supervised Learning): A specialized block optimized for recognizing specific words or short phrases in audio streams.

Example Model: A wake-word engine that detects "Hey Device" or "Start" to trigger a micro-controller process.

2. Classifier Layers (Visual Mode)

- Dense (Fully Connected) Layer: Every neuron is connected to all neurons in the previous layer. It is typically used at the end of the network to combine learned features and output the final classification probabilities.
- Conv1D (1D Convolutional) Layer: Applies a one-dimensional filter over the input. It is highly effective for finding local patterns in sequential or time-series data, such as accelerometer readings or audio waveforms.
- Conv2D (2D Convolutional) Layer: Applies a two-dimensional filter over the input to extract spatial features, such as edges, corners, and textures in image data.
- Dropout Layer: Randomly sets a specified percentage of neuron outputs to zero during the training phase. This prevents the model from relying too heavily on specific pathways, effectively mitigating overfitting.
- Flatten Layer: Converts multi-dimensional tensor outputs into a single, 1-dimensional vector so they can be processed by a Dense layer.
- Reshape Layer: Alters the dimensional shape of the data without changing its actual values. This is used to ensure the output of one layer matches the strict input requirements of the subsequent layer.

3. Model Compression Options (Deployment)

- Quantization (Int8): Converts the standard unoptimized 32-bit floating-point weights and activations (Float32) into 8-bit integers (Int8). This drastically reduces the model's storage size and speeds up mathematical operations on resource-constrained devices.
- EON (Edge Optimized Neural) Compiler: Translates the neural network directly into raw, optimized C++ source code instead of utilizing a runtime interpreter. This eliminates interpreter overhead, saving significant Flash and RAM space.
- EON Compiler (RAM Optimized): An aggressive compiler setting that takes standard EON compilation a step further to specifically minimize the runtime RAM footprint wherever algorithmically possible.

4. Synthetic Data Generation Modalities

- Image Data: Utilizes the DALL-E or FLUX Pro generation blocks to create highly realistic images based on user-provided text prompts.
- Audio Data: Utilizes the OpenAI Whisper keyword spotting block to generate realistic human speech variations of specific words from text.
- Time-Series Data: Utilizes Edge Impulse's proprietary Time-series Data Augmentation block to generate mathematical variations of existing sensor data samples.

Importance of Synthetic Data:

In machine learning applications, acquiring robust, real-world data is often the most expensive and time-consuming phase of development. Synthetic data allows developers to quickly generate vast amounts of labeled data, artificially filling gaps in under-represented classes. It is particularly crucial for simulating edge cases, dangerous scenarios, or rare environmental conditions that are virtually impossible to record in the real world, ultimately leading to a more robust, generalized, and accurate model.